

COP4533 - Programming Assignment 3

Bailey Watkins (UFID: 97706540)

Patrick McCormack (UFID: 73709580)

4/1/2026

Readme

Environment Setup

- Run `make init` if not using `uv`. Otherwise, continue to running the code.

Running the code

- Run `make run` (python venv) or `make run-uv` (uv) to run the example from the PDF.
- Run `make run assets/<file>` (python venv) or `make run-uv assets/<file>` if you wish to run the code with a provided input file. You may place any input files into the assets folder and run the code using them by providing the file name.

Compiling the README

- Run `make doc` to use Pandoc and $\text{\LaTeX}$ to create a PDF version of the README.

Written Component

Question 1: Empirical Comparison

the 10 input files along with the graph is inside the `src/assets` folder.

Question 2: Recurrence Equation

$$\begin{cases} 0 & \text{if either string has a length of 0} \\ \max(v_i + \text{OPT}(i-1, j-1), \text{OPT}(i-1, j), \text{OPT}(i, j-1)) & \text{if there is a match: } A[i] = B[j] \rightarrow v(A[i]) = v(B[j]) \\ \max(\text{OPT}(i-1, j), \text{OPT}(i, j-1)) & \text{if there's no match: } A[i] \neq B[j] \end{cases}$$

This recurrence equation is correct because:

- a) the base case only needs to consider if either string is empty, which immediately renders the matching and therefore max value to 0
- b) the matching case chooses either the character and its value, or it skips one char or the other, leading to 2 smaller subproblems (max used for taking the best value for optimality)
- c) since there are no matches, the program must skip one char or the other. This leads to 2 subproblems, one for $i-1$ and the other for $j-1$. (max used for taking the best value for optimality)

The use of 'max' gives each subproblem in OPT the max (and therefore best) possible value for the matchings between A and B.

Building and reusing these optimized subproblems will give the global optimal solution.

Question 3: Big-Oh

```
HVLCs(A, B):
    m = length(A)
    n = length(B)

    init table/array of size m by n (call it M)

    for i from 1 to m:
        for j from 1 to n:
            if A[i-1] == B[j-1]:
                M[i][j] = M[i-1][j-1] + val(A[i-1])
            else:
                M[i][j] = max(M[i-1][j], M[i][j-1])
    return M[m][n]
```

Runtime: $O(m \cdot n)$